Manipulación de Cadenas y Punteros en C/C++: Control de Acceso Mediante Desreferenciación en Memoria RAM

Malek Cheheid

Laboratorio de Programación

5.º Año 3.º División «A»

Profesor: Yamil Ganduglia

E.E.S.T. N.º 1 «Eduardo Ader» — Vicente López

Octubre de 2026

Repositorio: https://github.com/malekcheheid-art/CADENASPUNTEROS

Manipulación de Cadenas y Punteros en C/C++: Control de Acceso Mediante Desreferenciación en Memoria RAM

Para entender la programación de bajo nivel hay que ver lo que realmente pasa dentro de la memoria RAM. Eso es lo que hice en esta actividad: en lugar de tratar las variables como cajas negras, las entendí como casilleros físicos de la memoria, cada uno con un número que lo identifica. Ese número es su dirección. El trabajo consistió en escribir un programa en C++ que controla el acceso por edad, pero lo que realmente se aprende es cómo el lenguaje permite llegar a un casillero concreto de la memoria y modificar lo que hay adentro.

Para hacerlo usé dos operadores que sostienen todo el funcionamiento de los punteros: el operador de dirección (**&**), que devuelve la ubicación física de una variable, y el operador de desreferenciación (*****), que me deja leer o cambiar lo que hay en esa ubicación. Organicé el proyecto en un repositorio de Git, compilé el código y lo ejecuté en la consola para comprobar que funciona como explica la teoría.

Objetivos

Objetivo General

Desarrollé un sistema básico de control de acceso por edad en C++ que maneja los datos directamente en la memoria RAM usando punteros y desreferenciación, para entender cómo se comporta la memoria por dentro mientras un programa está corriendo.

Objetivos Específicos

a) Identifiqué el contenido y la ubicación de una variable dentro de la memoria RAM.

b) Diferencié el uso del operador de dirección (&) del operador de desreferenciación (*).

c) Capturé datos desreferenciando un puntero, de modo que el valor quede guardado en el casillero de la variable original.

d) Representé e interpreté direcciones de memoria en notación hexadecimal.

e) Analicé, compilé y ejecuté el programa en la consola, comprobando las dos ramas: acceso aprobado y acceso rechazado.

f) Documenté todo el proceso en este informe, escrito en Normas APA v7.

Marco Teórico

Memoria RAM

La memoria de acceso aleatorio (RAM, por sus siglas en inglés) es el espacio físico donde se guardan las variables mientras el programa está corriendo. No es un depósito pasivo: el microprocesador la está usando todo el tiempo. Cada dato ocupa un casillero, y cada casillero tiene una dirección única que lo identifica sin ambigüedad. Cuando el programa termina, lo que había en la RAM deja de tener sentido, y por eso se la llama almacenamiento temporal.

Dirección Hexadecimal

Una dirección de memoria es, en el fondo, un número. El sistema operativo no la escribe en base decimal, sino en base 16 (hexadecimal), y le pone el prefijo «0x» delante. Esa decisión

tiene razones técnicas concretas.

La primera es que el hexadecimal ocupa menos lugar. Un byte vale como máximo 255 y en hexadecimal se escribe con dos dígitos (por ejemplo, 0xFF), mientras que en decimal a veces hacen falta tres. La segunda es que las direcciones están alineadas a potencias de dos —2, 4, 8 o 16 bytes—, y esas cantidades se escriben muy fácil en base 16. La tercera, y la más importante, es que cada dígito hexadecimal equivale exactamente a cuatro bits, así que pasarlo a binario es directo. La dirección 0x61fe1c, que aparece como ejemplo en la consigna, tiene seis dígitos, o sea 24 bits, y eso permite direccionar 16 MB de memoria.

Puntero

Un puntero es una variable que guarda la dirección de memoria de otra variable. Se declara con un asterisco entre el tipo y el nombre, como en **int* p;**. Lo importante es esto: p no guarda la edad del usuario, guarda la ubicación de la variable que sí la guarda. El valor que tiene p —por ejemplo, 0x61FE10— es una dirección, no una edad.

Operador de Dirección (&)

El operador de dirección se escribe con un ampersand (&). Devuelve la dirección física en la RAM donde está guardada una variable. La expresión **&edad** no me da la edad: me da la dirección del casillero que la contiene. Es la operación contraria a la desreferenciación, y las dos se usan juntas al crear el puntero: **int* p = &edad;** quiere decir «creá p y guardá adentro la dirección de edad».

Desreferenciación (*)

La desreferenciación es la operación que me deja acceder al valor guardado en la dirección apuntada. El asterisco aparece en dos lugares distintos con efectos distintos. En una declaración, como **int* p**, dice que p es un puntero. En una expresión, como ***p**, accede al contenido de la dirección. Esta segunda forma es la que me permite leer y, sobre todo, cambiar la variable original desde cualquier parte del programa.

Glosario

La consigna pide dominar un glosario de conceptos técnicos. Estas son las definiciones:

Tabla 1

Glosario de conceptos técnicos

Término	Definición
Memoria RAM	Espacio físico de almacenamiento temporal donde residen las variables durante la ejecución del programa.
Dirección hexadecimal	Representación numérica en base 16 (que inicia con 0x) para identificar un casillero exacto de la RAM.
Puntero (pointer)	Variable que guarda la dirección de memoria de otra variable del sistema.
Operador de dirección (&)	Operador que devuelve la dirección física en la RAM donde está guardada una variable.
Desreferenciar (*)	Operación para leer o modificar el valor contenido en la dirección de memoria apuntada.

Nota. **RAM** = Random Access Memory (memoria de acceso aleatorio).

Metodología

Enfoque y Entorno de Trabajo

Trabajé de forma deductiva: primero estudié la teoría de la memoria y de los punteros en C++, después escribí un programa que mostrara esos conceptos, y por último lo ejécuté en la consola para comprobar los resultados. Usé Visual Studio Code con su terminal integrada y el compilador GNU g++ (tdm64) versión 10.3.0 sobre Windows. Pude compilar en mi propia computadora, así que no necesité trabajar con OnlineGDB.

Estructura del Proyecto

Armé el proyecto siguiendo la estructura de carpetas que pide la consigna.

```
CADENASPUNTEROS/
├── .gitignore                  # Configuración de editores ignorada por Git
├── LICENSE                     # Licencia MIT (Uso escolar)
├── README.md                   # Explicación del uso de punteros en la RAM
├── docs/
│   └── InformeProyectoCADENASPUNTEROS.pdf   # ENTREGABLE (Informe APA v7)
├── src/
│   └── main.cpp                # Código fuente del programa en C++
└── capturas/
    ├── ejecucion_punteros.png  # Terminal con direcciones hexadecimales
    └── esquema_ram.png         # Diagrama de la memoria RAM
```

Código Fuente

El programa hace un control de acceso por edad. Se apoya en dos cosas: capturar el dato desreferenciando el puntero, y la condicional que evalúa el valor guardado en memoria. Abajo muestro las partes más importantes; el código entero está en el Apéndice A.

```
int edad = 0;
int* p = &edad;      // p guarda la dirección de memoria física de 'edad'

cin >> *p;           // captura el dato guardándolo directo en la memoria

if (*p >= 18) {
    // se muestra la edad con *p y la dirección con p
```

```
}
```

Tomé tres decisiones al escribirlo. Primero, siempre leo el dato con ***p**, incluso cuando lo muestro en pantalla, así me aseguro de que el valor que veo es el que está realmente en el casillero original y no una copia. Segundo, cuando quiero mostrar la dirección uso **p** sin desreferenciar, porque lo que interesa ver es justamente la dirección. Tercero, usé el número 18 como límite de mayoría de edad dentro de la condicional.

Resultados

Compilación

Compilé el programa sin errores ni advertencias, usando el comando que indica la consigna para Windows.

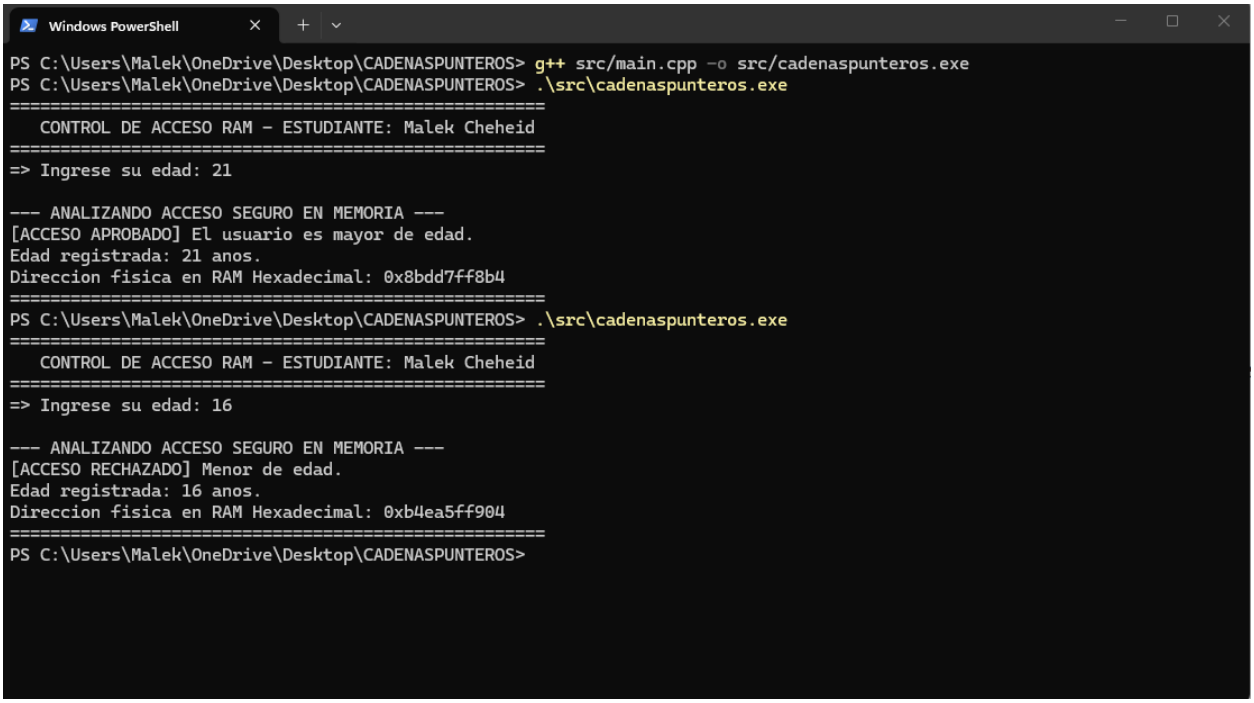
```
g++ src/main.cpp -o src/cadenaspunteros.exe
```

Ejecución del Programa

Ejecuté el programa dos veces para recorrer las dos ramas de la condicional. La primera vez, con 21 años, el acceso fue aprobado. La segunda, con 16 años, fue rechazado.

Figura 1

Ejecución del programa en la terminal



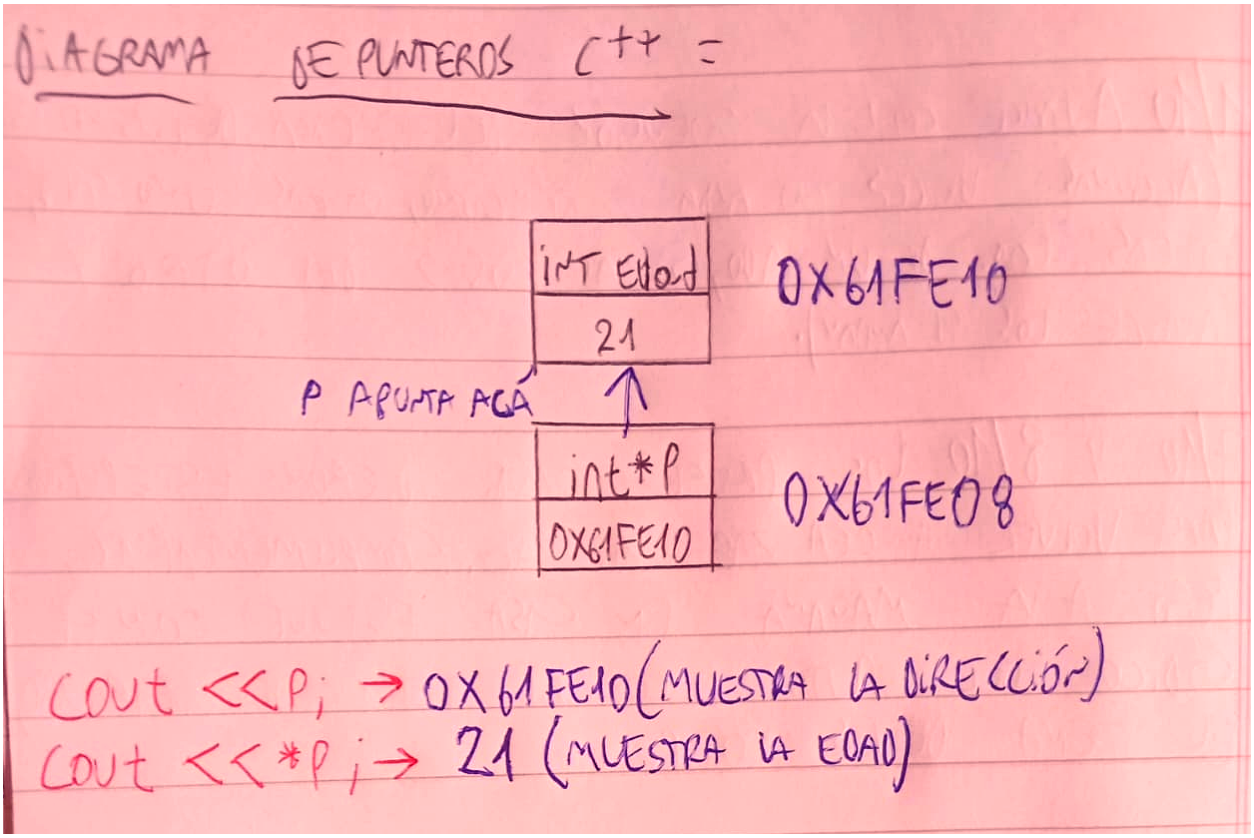
Se nota que el valor de la dirección cambia entre ejecuciones. Esto es normal y no es un error del programa: la dirección de un casillero en la pila depende del estado previo de la memoria, y el sistema operativo no garantiza que sea la misma. Lo que sí se comprueba siempre es que p tenga una dirección real y válida.

Esquema de la Memoria RAM

Para ver con claridad el vínculo entre la variable y el puntero, dibujé a mano el esquema de memoria de la Figura 2, donde se ve la dirección de cada casillero, el valor guardado y la relación de puntero.

Figura 2

Esquema de la memoria RAM



Cuestionario Técnico de Control

La consigna plantea tres preguntas sobre cómo funcionan los punteros en la memoria. Estas son mis respuestas.

Pregunta 1: Diferencia Entre & y *

¿Qué valor imprime en pantalla la instrucción cout << p; comparada con cout << *p;?

Los dos operadores van en direcciones opuestas. El operador de dirección (&) parte de la variable y va hacia su ubicación: devuelve la dirección física del casillero. El operador de desreferenciación (*) hace el camino inverso: va de la dirección a la variable y me entrega lo que hay adentro. Una forma fácil de recordarlo: & dice dónde está el dato, y * dice qué hay en ese lugar.

cout << p; muestra la **dirección de memoria** en notación hexadecimal, es decir, el dato guardado en el casillero del propio puntero. En la ejecución de la Figura 1 aparece 0x8bdd7ff8b4. Este valor cambia entre ejecuciones porque depende del estado de la pila.

cout << *p; muestra el **valor de la edad** que está guardado en la dirección apuntada. En la misma ejecución aparece 21 anos. Este valor sí es siempre el mismo, porque depende del dato que escribí y no del estado de la memoria.

En resumen: **p** responde «¿dónde está el dato?» y devuelve una dirección. ***p** responde «¿qué dato hay en esa dirección?» y devuelve el contenido. Gracias a esa diferencia el programa puede mostrar las dos cosas al mismo tiempo: la edad del usuario y la dirección física donde está guardada, que es justo lo que busca el ejercicio.

Pregunta 2: Error Crítico al Guardar Entrada

¿Por qué es obligatorio desreferenciar el puntero (*p) al usar cin para modificar el valor de la variable original?

No es una cuestión de estilo: es el sistema de tipos del lenguaje. La variable p está declarada como **int***, así que la expresión p es de tipo «puntero a entero». La operación **operator**>> de la clase **istream** no tiene ninguna versión que acepte un **int*** como destino de

escritura: las que existen aceptan referencias a números u objetos, nunca punteros.

Lo verifiqué compilando. Al cambiar la instrucción correcta por la incorrecta, el compilador rechaza el código así:

```
error: no match for 'operator>>' (operand types are
                        'std::istream' and 'int*')
```

Entonces **cin >> p;** no es un error que aparece al ejecutar el programa: es un error de compilación, y el programa ni siquiera llega a construirse. Esa es justamente la protección que da el lenguaje. Si me olvido del asterisco, lo que podría pasar —sobrescribir la dirección física con un número cualquiera— queda imposible, porque el compilador se niega a generar ese código.

La razón más profunda es que el asterisco cambia el tipo de la expresión. La expresión p es de tipo **int***, o sea una dirección, y una dirección no puede ser destino de una escritura de datos. En cambio, ***p** es de tipo **int**: es una referencia al casillero apuntado, y ese es justamente el tipo que cin sabe escribir. Por eso la forma correcta es **cin >> *p;**: el asterisco le dice al compilador «entrá al casillero apuntado y escribí ahí», en vez de cambiar la dirección guardada en el puntero.

Pregunta 3: Formato Hexadecimal

¿Por qué el sistema operativo utiliza la notación hexadecimal en lugar de decimal para representar las direcciones de memoria?

Hay cuatro razones, y todas son técnicas.

a) Correspondencia directa con el binario. Como ya expliqué, cada dígito hexadecimal son exactamente cuatro bits. Por eso pasar la dirección a binario —que es como el procesador realmente las maneja— es inmediato, sin cuentas en el medio. 0x61FE10 se traduce sin esfuerzo a 0110 0001 1111 1110 0001 0000.

b) Alineación a potencias de dos. Las direcciones se alinean a 2, 4, 8 o 16 bytes, o sea a potencias de dos. Como la base 16 también es una potencia de dos ($16 = 2^4$), los múltiplos y los límites de bloque se escriben enseguida en hexadecimal. Los últimos dígitos de una dirección muestran la alineación: 0x…F0 está alineado a 16 bytes y 0x…FC a 4 bytes.

c) Es más corto de leer. Una dirección de 64 bits ocupa 16 dígitos hexadecimales, mientras que en decimal necesitaría hasta 20. Como buena parte de lo que leo al analizar un volcado de memoria o un desensamblado son direcciones y números, esta notación corta hace el trabajo mucho más llevadero.

d) Está más pegada al hardware. Los procesadores guardan y trabajan con direcciones en binario. Cuando el sistema operativo me muestra esa información, elige la representación que más se parece a la estructura binaria real, y eso ayuda a diagnosticar problemas de direccionamiento y de segmentos de memoria.

Conclusiones

La actividad me permitió llegar al objetivo. Escribí un sistema de control de acceso por edad en C++ que trabaja directamente sobre la memoria RAM con punteros y desreferenciación, y así comprobé de forma verificable lo que dice la teoría.

Confirmé que & y * hacen cosas distintas: uno me da la ubicación de la variable y el otro me da lo que hay guardado ahí. También comprobé, compilando, que el asterisco es obligatorio para leer datos: si lo saco, el programa ni siquiera se construye.

Me quedé con una regla práctica: un puntero no contiene el dato, contiene la ubicación del dato. Toda la lógica del programa sale de respetar esa diferencia, y el error más común —olvidar el asterisco— es justamente el intento de tratar al puntero como si fuera la variable a la que apunta.

Referencias

cplusplus.com. (s. f.). *Pointers* [Tutorial]. https://www.cplusplus.com/doc/tutorial/pointers/

GeeksforGeeks. (2026). *C++ Pointers*. https://www.geeksforgeeks.org/cpp/cpp-pointers/

Microsoft. (2016). *Operador de dirección: &* (Referencia del lenguaje C++). Microsoft Learn. https://learn.microsoft.com/es-es/cpp/cpp/address-of-operator-amp?view=msvc-170

Microsoft. (2016). *Operador de direccionamiento indirecto:* * (Referencia del lenguaje C++). Microsoft Learn. https://learn.microsoft.com/es-es/cpp/cpp/indirection-operator-star?view=msvc-170

Apéndice A

Código Fuente Completo (src/main.cpp)

Abajo está el contenido entero del archivo **src/main.cpp**, tal como quedó en el repositorio.

```
/*
   ========================================================================
   Materia: Laboratorio de Programación (LPR)
   E.E.S.T. N° 1 "Eduardo Ader" – Vicente López
   Curso: 5° Año 3° División "A"
   Profesor: Prof. York
   Archivo: main.cpp
   Objetivo: Control de accesos operando directamente sobre la RAM con punteros.
   ========================================================================
*/

#include <iostream>

using namespace std;

int main() {
    int edad = 0;
    int* p = &edad; // El puntero 'p' guarda la dirección de memoria física de 'edad'

    // ========================================================================
    // REGLA OBLIGATORIA (EVITA EL PLAGIO):
    // Modifiquen la salida agregando su Nombre y Apellido reales.
    // ========================================================================

    cout << "=====================================================" << endl;
    cout << "   CONTROL DE ACCESO RAM - ESTUDIANTE: Malek Cheheid " << endl;
    cout << "=====================================================" << endl;

    cout << "=> Ingrese su edad: ";

    // ========================================================================
    // 1. CAPTURA MEDIANTE DESREFERENCIACIÓN (CÓDIGO EXPLICADO)
    //  EXPLICACIÓN TÉCNICA:
    //    Usamos 'cin >> *p;' porque el asterisco '*' desreferencia al puntero.
    //    Esto le indica al compilador que debe guardar el número ingresado
```

```
    //    directamente en el casillero de memoria original de 'edad'.
    //    Si pusiéramos 'cin >> p;' sin asterisco, intentaríamos sobrescribir
    //    la dirección hexadecimal física de la RAM, provocando un error grave.
    // ========================================================================

    cin >> *p; // Captura el dato guardándolo directo en la memoria apuntada


    cout << "\n--- ANALIZANDO ACCESO SEGURO EN MEMORIA ---" << endl;

    // ========================================================================
    // 2. CONDICIONAL CON PUNTERO DESREFERENCIADO
    //  EXPLICACIÓN TÉCNICA:
    //    Evaluamos el valor alojado en la memoria usando '*p >= 18'.
    //    Si mostramos solo 'p', la consola imprimirá la dirección hexadecimal (ej: 0x61fe1c).
    // ========================================================================

    if (*p >= 18) {
        cout << "[ACCESO APROBADO] El usuario es mayor de edad." << endl;
        cout << "Edad registrada: " << *p << " anos." << endl;
        cout << "Direccion fisica en RAM Hexadecimal: " << p << endl;
    } else {
        cout << "[ACCESO RECHAZADO] Menor de edad." << endl;
        cout << "Edad registrada: " << *p << " anos." << endl;
        cout << "Direccion fisica en RAM Hexadecimal: " << p << endl;
    }

    cout << "======================================================" << endl;
    return 0; // Finalización exitosa de Windows (Código 0)
}
```